

Введение

Данное задание является очень простым примером реверс-инжиниринга. При запуске задания мы получаем бинарный файл `ch1.bin`. Цель — найти правильную строку, которая позволит пройти внутреннюю проверку программы.

В этом руководстве мы разберём процесс анализа от определения типа файла до понимания логики работы программы.

Инструменты

Для решения используются следующие инструменты:

- `file` — определение типа бинарного файла
- `strings` — извлечение текстовых строк
- Cutter, Ghidra или radare2 — декомпиляция и анализ кода
- `ltrace` — трассировка библиотечных вызовов (опционально)

Пошаговое прохождение

Шаг 1. Определение типа файла

Сначала проверим тип бинарного файла с помощью утилиты `file`. Файл оказывается **динамически слинкованным ELF-бинарником и не застриппованным** (символы отладки сохранены). Это упрощает дальнейший анализ.

```
$ file ch1.bin
ch1.bin: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, for GNU/Linux 2.6.9, not stripped
```

Шаг 2. Быстрый поиск с помощью `strings`

Применяем утилиту `strings` к файлу `ch1.bin`. Среди извлечённых строк сразу обнаруживается строка `123456789`. Она выглядит как возможный правильный ввод.

Однако, чтобы понять, как именно программа обрабатывает ввод и почему эта строка работает, перейдём к полноценному анализу кода.

Шаг 3. Анализ в декомпиляторе

Открываем файл в Cutter (или в Ghidra / radare2) и переходим в раздел декомпилятора.

```

undefined4 main(void)
{
    undefined4 uVar1;
    int32_t iVar2;
    void *pvStack20;
    char *s1;
    char *var_8h;

    puts(0x804884c);
    puts("##      Bienvenue dans ce challenge de cracking      ##");
    puts("#####\n");
    printf("Veuillez entrer le mot de passe : ");
    uVar1 = getString(pvStack20);
    iVar2 = strcmp(uVar1, "123456789");
    if (iVar2 == 0) {
        printf("Bien joue, vous pouvez valider l'epreuve avec le pass : %s!\n", "123456789");
    } else {
        puts("Dommage, essaye encore une fois.");
    }
    return 0;
}

```

В коде программы видно следующую логику:

- Программа считывает пользовательский ввод и сохраняет его в локальную переменную.
- Далее вызывается функция strcmp, которая сравнивает введённую строку с жёстко заданной строкой 123456789.
- Результат сравнения сохраняется в другую переменную.
- Если результат strcmp равен 0 (строки полностью совпадают), условие if обходится и программа считает ввод правильным.

Таким образом, правильным паролем является именно строка 123456789.

Шаг 4. Подтверждение с помощью ltrace (опционально)

Для наглядности можно запустить программу под трассировщиком ltrace. Это позволит увидеть реальные вызовы библиотечных функций, в том числе strcmp, и убедиться, что сравнение происходит именно с указанной строкой.

```

ltrace ./ch1.bin
__libc_start_main(0x804869d, 1, 0xffbcd2f4, 0x8048750 <unfinished ...>
puts("#####\n")
puts("##      Bienvenue dans ce cha...##      Bienvenue dans ce challenge de cracking      ##")
puts("#####\n")
printf("Veuillez entrer le mot de passe *...")
malloc(2)
getchar(2, 0xf7f5b000, 0xffbcd238, 0xf7dca085)Veuillez entrer le mot de passe : 123456789
realloc(0x944b5b0, 2)
getchar(0x944b5b0, 2, 0xffbcd238, 0xf7dca085)
realloc(0x944b5b0, 3)
getchar(0x944b5b0, 3, 0xffbcd238, 0xf7dca085)
realloc(0x944b5b0, 4)
getchar(0x944b5b0, 4, 0xffbcd238, 0xf7dca085)
realloc(0x944b5b0, 5)
getchar(0x944b5b0, 5, 0xffbcd238, 0xf7dca085)
realloc(0x944b5b0, 6)
getchar(0x944b5b0, 6, 0xffbcd238, 0xf7dca085)
realloc(0x944b5b0, 7)
getchar(0x944b5b0, 7, 0xffbcd238, 0xf7dca085)
realloc(0x944b5b0, 8)
getchar(0x944b5b0, 8, 0xffbcd238, 0xf7dca085)
realloc(0x944b5b0, 9)
getchar(0x944b5b0, 9, 0xffbcd238, 0xf7dca085)
realloc(0x944b5b0, 10)
getchar(0x944b5b0, 10, 0xffbcd238, 0xf7dca085)
strcmp("123456789", "123456789")
printf("Bien joue, vous pouvez valider l'... , "123456789"Bien joue, vous pouvez valider l'epreuve avec le pass : 123456789!
+++ exited (status 0) +++

```

Шаг 5. Получение решения

При вводе строки 123456789 условие проверки успешно обходится.